

INFORME TÉCNICO

AeroData

Plataforma web para la exploración y análisis
de datos de calidad del aire

DOMINIO DE DATOS

Calidad del aire / clima

FUENTE DE DATOS

Open-Meteo Air Quality API (abierta)

BASE DE DATOS

PostgreSQL (Neon)

DESPLIEGUE

Vercel

FECHA

16 de agosto de 2026

Índice

1. Resumen ejecutivo
2. Objetivos del proyecto
3. Arquitectura general del sistema
4. Fuente de datos: API pública
5. Proceso ETL (Extracción, Transformación y Carga)
6. Modelo de datos y almacenamiento
7. Capa de API y visualizaciones
8. Despliegue en la nube (Vercel)
9. Resultados obtenidos
10. Conclusiones y trabajo futuro

1. Resumen ejecutivo

AeroData es una aplicación web full-stack que permite explorar y analizar un conjunto significativo de datos de calidad del aire de ciudades de todo el mundo. El sistema identifica y consume una fuente de datos abierta (Open-Meteo Air Quality API), ejecuta un proceso ETL que normaliza y enriquece la información, la almacena de forma estructurada en una base de datos PostgreSQL y la presenta mediante un dashboard analítico con visualizaciones interactivas. La aplicación está preparada para desplegarse en Vercel, con una actualización automática de datos programada mediante cron.

En su estado actual, el sistema gestiona 2304 lecturas horarias correspondientes a 12 ciudades, con un Índice de Calidad del Aire (AQI) promedio de 74 puntos. Estos datos se refrescan de forma incremental e idempotente en cada ejecución del ETL.

2. Objetivos del proyecto

- Identificar y consumir fuentes de datos abiertos o APIs públicas.
- Implementar un proceso básico de ETL (Extracción, Transformación y Carga).
- Almacenar y gestionar los datos de forma estructurada en una base de datos relacional.
- Generar visualizaciones interactivas que faciliten el análisis exploratorio.
- Desplegar la aplicación en la nube utilizando Vercel.
- Documentar correctamente el proyecto y su arquitectura.

3. Arquitectura general del sistema

La solución sigue una arquitectura por capas sobre Next.js (App Router), donde el mismo proyecto contiene tanto el backend (rutas de API y lógica ETL) como el frontend (dashboard). El flujo de datos es unidireccional: la API pública alimenta el ETL, este persiste en PostgreSQL y las rutas de consulta sirven los datos ya agregados al cliente.

Open-Meteo API % % %° ETL (extract/transform/load) % % %° PostgreSQL (Neon)
%
Rutas API (/api/summary, /api/readings)
%
Dashboard React (SWR + Recharts)

Componentes principales

Capa	Tecnología	Responsabilidad
Ingesta	fetch + Open-Meteo	Extracción de datos horarios
ETL	TypeScript (lib/etl.ts)	Transformación y carga idempotente
Persistencia	PostgreSQL / Neon	Almacenamiento estructurado
API	Next.js Route Handlers	Consultas agregadas
Frontend	React, SWR, Recharts	Visualización interactiva
Planificación	Vercel Cron	Actualización automática diaria

4. Fuente de datos: API pública

Se utiliza la Open-Meteo Air Quality API, un servicio de datos abiertos que no requiere clave de API y ofrece mediciones horarias de contaminantes atmosféricos por coordenadas geográficas. Para cada ciudad del catálogo se solicitan las series temporales de los principales contaminantes.

Variables consumidas

- PM2.5 y PM10 - material particulado ($\mu\text{g}/\text{m}^3$).
- Ozono (O3), dióxido de nitrógeno (NO2), dióxido de azufre (SO2) y monóxido de carbono (CO).
- Índice de calidad del aire europeo y estadounidense (US AQI).

El catálogo incluye 12 ciudades representativas de distintos continentes, lo que permite comparativas geográficas significativas.

5. Proceso ETL (Extracción, Transformación y Carga)

El núcleo del proyecto reside en lib/etl.ts, que implementa las tres fases del proceso de forma explícita:

Extracción (Extract)

Se realizan peticiones HTTP concurrentes a la API para cada ciudad, recuperando las series horarias del rango disponible. Se controla el resultado de cada petición para tolerar fallos parciales sin abortar el proceso completo.

Transformación (Transform)

- Normalización de unidades y de la estructura temporal (una fila por ciudad y hora).
- Cálculo y clasificación del Índice de Calidad del Aire (AQI) en categorías: Buena, Moderada, Dañina para grupos sensibles y Dañina.
- Descarte de registros incompletos o inválidos antes de la carga.

Carga (Load)

Los registros transformados se insertan en PostgreSQL mediante operaciones idempotentes (upsert por clave ciudad + marca temporal), de modo que ejecutar el ETL varias veces no genera duplicados. Cada ejecución registra su resultado (filas cargadas, ciudades y duración) para trazabilidad.

```
INSERT INTO air_quality_readings (...)  
VALUES (...)  
ON CONFLICT (city_id, recorded_at) DO UPDATE SET ...;
```

6. Modelo de datos y almacenamiento

Los datos se almacenan de forma estructurada en dos tablas relacionales. El esquema se crea automáticamente en el primer arranque de la aplicación (ensureSchema), lo que simplifica el aprovisionamiento.

Tabla: cities

Columna	Tipo	Descripción
id	serial PK	Identificador interno
slug	text unique	Clave legible (p. ej. madrid)
name	text	Nombre de la ciudad
country	text	País
latitude / longitude	double	Coordenadas geográficas

Tabla: air_quality_readings

Columna	Tipo	Descripción
id	serial PK	Identificador de lectura
city_id	int FK	Referencia a cities
recorded_at	timestamp tz	Marca temporal de la medición
pm2_5 / pm10	double	Material particulado ($\mu\text{g}/\text{m}^3$)
ozone / no2 / so2 / co	double	Concentración de gases
us_aqi	int	Índice de calidad del aire
category	text	Clasificación cualitativa

7. Capa de API y visualizaciones

Endpoints

Ruta	Método	Función
/api/summary	GET	KPIs y última lectura por ciudad
/api/readings	GET	Series temporales filtrables
/api/etl	POST	Disparo manual del ETL
/api/cron/etl	GET	Ejecución programada (Vercel Cron)

Visualizaciones del dashboard

- Tarjetas KPI: ciudades monitoreadas, AQI promedio global, aire más limpio y mayor contaminación.
- Gráfico de evolución temporal con selector de ciudad y contaminante.
- Ranking comparativo de ciudades por calidad del aire.

- Desglose de contaminantes de la lectura más reciente.
- Tabla de lecturas recientes con clasificación por color.

El frontend usa SWR para la obtención y cacheo de datos en cliente, y Recharts para las visualizaciones. La interfaz es responsive y sigue un diseño analítico moderno.

8. Despliegue en la nube (Vercel)

La aplicación se despliega en Vercel directamente desde el proyecto. La conexión a la base de datos se gestiona mediante la variable de entorno DATABASE_URL (PostgreSQL de Neon), configurada en el panel de variables del proyecto.

Actualización automática (Cron)

El archivo vercel.json define una tarea programada que ejecuta el ETL de forma automática. En el plan Hobby (gratuito) de Vercel los cron jobs se ejecutan una vez al día; la planificación se estableció por tanto a una ejecución diaria (0 6 * * *). El endpoint de cron valida un secreto opcional (CRON_SECRET) para evitar ejecuciones no autorizadas.

```
{ "crons": [ { "path": "/api/cron/etl", "schedule": "0 6 * * *" } ] }
```

9. Resultados obtenidos

Tras la ejecución del ETL, el sistema gestiona 2304 lecturas de 12 ciudades. La siguiente tabla muestra la última lectura registrada por ciudad, ordenada de mayor a menor contaminación:

Ciudad	País	AQI	PM2.5	PM10	Categoría
Pekín	China	170	63.8	71.5	Dañina
Nueva Delhi	India	161	49.8	81.5	Dañina
Santiago	Chile	87	34	34.1	Moderada
Nueva York	EE. UU.	66	14.1	15.7	Moderada
Barcelona	España	60	14.7	24	Moderada
Madrid	España	53	10.5	17	Moderada
Ciudad de México	México	53	4.8	5.5	Moderada
Buenos Aires	Argentina	53	8.1	8.2	Moderada
Bogotá	Colombia	52	6.2	6.2	Moderada
París	Francia	49	5.6	9.2	Buena
Lima	Perú	47	11.2	46.7	Buena
Londres	Reino Unido	37	7.6	10.7	Buena

Se observa un claro contraste geográfico: las ciudades asiáticas (Pekín, 170; Nueva Delhi, 161) presentan los niveles más altos de contaminación, mientras que ciudades europeas como Londres (37)

registran la mejor calidad del aire del conjunto.

10. Conclusiones y trabajo futuro

El proyecto cumple íntegramente los objetivos planteados: consume una fuente de datos abierta, implementa un pipeline ETL completo, almacena la información de forma estructurada en PostgreSQL, ofrece visualizaciones interactivas y está preparado para su despliegue en Vercel con actualización automática.

Líneas de trabajo futuro

- Incorporar un mapa geográfico interactivo de las ciudades monitoreadas.
- Ampliar el catálogo de ciudades y el histórico de datos.
- Añadir alertas y umbrales configurables por contaminante.
- Incluir predicciones a corto plazo a partir de las series temporales.

